

L&T Edu Tech

Web Server Log Analytics and User Behaviour Monitoring

Course: Data Processing with Hive and Pig Latin

College: Sasi Institute of Technology and Engineering

Team Members:

1. Donka Renuka – 24K61A6110
2. Shaik Saara Kauser – 24K61A6143
3. Janga Durga Bhavani – 24K61A6114
4. Pachari Tanuja – 24K61A6134

Project Overview & Objectives

Objective:

Design a big data analytics solution to process large-scale web server logs using Hive and Pig. Analyse website traffic patterns, user navigation paths, error trends, and most visited pages.

Key Deliverables:

- Hive database and tables for log storage
- HiveQL queries for traffic analysis
- Pig scripts for log cleansing and transformation
- Dashboard/report showing website insights
- Performance optimization using partitioning and bucketing

TABLE OF CONTENTS

No	Content
1	Hadoop Setup
2	Hive Setup
3	Hive Queries Execution
4	Pig Latin Setup
5	Pig Latin Scripts Execution
6	Data Extraction & Visualizations
7	Dashboard & Results
8	Project Setup Guide & GitHub Resources

1.HADOOP SETUP

The Hadoop environment was configured and verified successfully before proceeding with the Hive and Pig Latin operations. The following screenshots document the important commands and steps used during the Hadoop setup process.

1.1 Starting Hadoop Services

Command:

1.start-all.sh

2.jps

The Hadoop services were started, and the jps command was used to verify that the required Hadoop processes were running successfully.

Figure 3: Starting Hadoop Services and Verifying Using JPS

```
hdp@ubuntu-24: ~$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hdp in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [ubuntu-24]
2026-09-09 07:33:45,248 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Starting resourcemanager
Starting nodemanagers
hdp@ubuntu-24: ~$ jps
3937 ResourceManager
3492 DataNode
4069 NodeManager
3352 NameNode
5929 Jps
3706 SecondaryNameNode
hdp@ubuntu-24: ~$
```

1.2 Creating the Project Directory

The project directory was created to organize the files and datasets required for the Hadoop, Hive, and Pig Latin operations.

Figure 1: Creating the Project Directory

Command: `hadoop fs -mkdir /project`

```
hdp@ubuntu-24: ~$ hadoop fs -mkdir /project
2026-09-09 07:42:42,435 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

1.3 Listing Hadoop Directories

Command: `hadoop fs -ls /`

The Hadoop directory structure was checked using the directory listing command to verify the availability of the required folders.

Figure 2: Listing Hadoop Directories

```
hdp@ubuntu-24: ~$ hadoop fs -ls /
2026-09-09 07:42:09,919 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 5 items
drwxr-xr-x - hdp supergroup 0 2026-09-07 16:22 /p
drwxr-xr-x - hdp supergroup 0 2026-04-28 13:16 /test
drwxr-xr-x - hdp supergroup 0 2026-09-07 16:32 /tmp
drwxr-xr-x - hdp supergroup 0 2026-09-08 09:50 /today
drwxr-xr-x - hdp supergroup 0 2026-09-07 16:23 /user
hdp@ubuntu-24: ~$
```

1.4 Copying Data from Local System to Hadoop

Command:

1.hadoop fs -copyFromLocal local_path /project

2.hadoop fs -ls /project

The required project data was transferred from the local system to the Hadoop Distributed File System (HDFS) for further processing using Hive and Pig Latin.

Figure 4: Copying Data from Local System to HDFS

```
hdp@Ubuntu-24: $ hadoop fs -copyFromLocal /home/hdp/Downloads/web_server_logs_5000.csv /project
2026-09-09 07:43:53,625 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
hdp@Ubuntu-24: $ hadoop fs -ls /project
2026-09-09 07:44:04,576 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r-- 1 hdp supergroup 935735 2026-09-09 07:43 /project/web_server_logs_5000.csv
hdp@Ubuntu-24: $
```

2.HIVE SETUP

After successfully configuring Hadoop and starting the required Hadoop services, Hive was initialized for processing and analysing the web server log dataset.

2.1 Starting Hive

Command: hive

hive>>

The Hive shell was started from the terminal using the Hive command. Once Hive was successfully initialized, the hive> prompt was displayed, indicating that Hive was ready to accept SQL-like queries.

Figure 5: Starting Hive and Opening the Hive Shell

```
hdp@Ubuntu-24:~$ hive
Hive Session ID = 869386ad-ec8c-4408-ae5-1e799270480f

Logging initialized using configuration in jar:file:/home/hdp/Downloads/apache-hive-3.1.2-bin/lib/hive-common-3.1.2.jar!/hive-log4j2.properties Async: true
Hive Session ID = 447d2835-57e8-4492-942b-38fa2f0dbca7
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
hive>
```

2.2 Creating and Selecting the Project Database

Command:

- 1.show databases;
- 2.create database project;
- 3.use project;
- 4.show tables;

The available databases were first listed to verify the Hive environment. A separate project database was then created for storing the tables related to the web server log analysis. The newly created database was selected using the USE command.

Figure 6: Listing Databases, Creating the Project Database and Selecting the Database

```
hive> show databases;
OK
default
reduka
Time taken: 0.088 seconds, Fetched: 2 row(s)
hive> create database project;
OK
Time taken: 0.362 seconds
hive> use project;
OK
Time taken: 0.044 seconds
hive> show tables;
OK
Time taken: 0.093 seconds
hive>
```

2.3 Creating the Webserver Logs Table

Query:

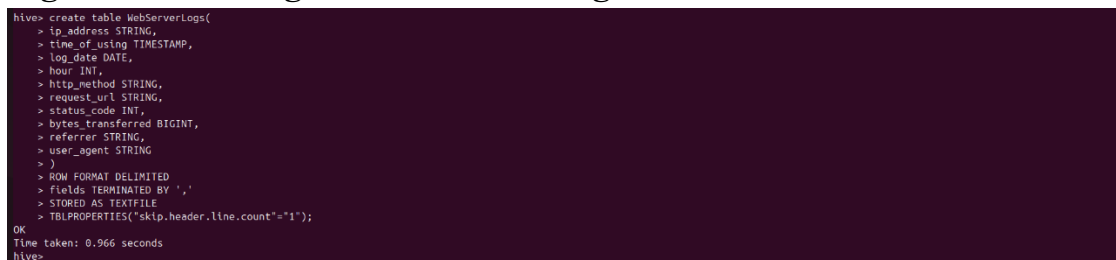
```
create table WebServerLogs(  
  ip_address STRING,  
  time_of_using TIMESTAMP,  
  log_date DATE,  
  hour INT,  
  http_method STRING,  
  request_url STRING,  
  status_code INT,  
  bytes_transferred BIGINT,  
  referrer STRING,  
  user_agent STRING  
)  
  
ROW FORMAT DELIMITED  
  
fields TERMINATED BY ','  
  
STORED AS TEXTFILE  
  
TBLPROPERTIES("skip.header.line.count"="1");
```

A Hive table named WebServerLogs was created to store the web server log data.

The table was defined with appropriate data types for each attribute. ROW FORMAT DELIMITED and FIELDS TERMINATED BY ',' were used because the input dataset is comma-separated. The table was configured to skip the first line of the dataset using:

```
TBLPROPERTIES("skip.header.line.count"="1")
```

Figure 7: Creating the WebServerLogs Hive Table



```
hive> create table WebServerLogs(  
  > ip_address STRING,  
  > time_of_using TIMESTAMP,  
  > log_date DATE,  
  > hour INT,  
  > http_method STRING,  
  > request_url STRING,  
  > status_code INT,  
  > bytes_transferred BIGINT,  
  > referrer STRING,  
  > user_agent STRING  
  > )  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY ','  
  > STORED AS TEXTFILE  
  > TBLPROPERTIES("skip.header.line.count"="1");  
OK  
Time taken: 0.966 seconds  
hive>
```

WebServerLogs Table Structure

The following table describes the columns used in the Hive table:

Column Name	Data Type	Description	Example Data
ip_address	STRING	IP address of the client	192.168.1.10
time_of_using	TIMESTAMP	Date and time of the request	2026-09-01 10:15:32
log_date	DATE	Date on which the request occurred	2026-09-01
hour	INT	Hour extracted from the request time	10
http_method	STRING	HTTP request method	GET
request_url	STRING	URL/resource requested by the client	/index.html
status_code	INT	HTTP response status code	200
bytes_transferred	BIGINT	Number of bytes transferred	5248
referrer	STRING	Previous webpage/source of the request	https://example.com
user_agent	STRING	Client/browser information	Mozilla/5.0

2.4 Loading Data from Hadoop into the Hive Table

Command:

```
LOAD DATA INPATH '/project/web_server_logs_5000.csv' INTO TABLE WebServerLogs;
```

The web server log dataset stored in HDFS was loaded into the WebServerLogs Hive table. This made the data available for querying and analysis using HiveQL.

Figure 8: Loading Data from HDFS into the Hive Table

```
hive> LOAD DATA INPATH '/project/web_server_logs_5000.csv' INTO TABLE WebServerLogs;
Loading data to table project.webserverlogs
OK
Time taken: 1.212 seconds
hive> |
```

2.5 Verifying the Loaded Data

Command:

```
SELECT * FROM WebServerLogs LIMIT 10;
```

After loading the dataset, the first 10 rows were displayed using a Hive query. This was used to verify that the data had been successfully loaded into the WebServerLogs table and that the records were being interpreted correctly.

Figure 9: Displaying the First 10 Rows of the Hive Table

```
hive> SELECT * FROM WebServerLogs LIMIT 10;
OK
172.16.0.20      2026-09-01 00:00:54      2026-09-01      0      GET      /products/item1 200      363      https://www.google.com Googlebot/2.1 (+http://www.google.com/bot.html)
10.0.0.11        2026-09-01 00:01:37      2026-09-01      0      GET      /static/style.css      200      4757      https://www.bing.com      "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple
WebKit/537.36 (KHTML
192.168.1.78     2026-09-01 00:07:05      2026-09-01      0      GET      /products      200      2839      https://www.facebook.com      Googlebot/2.1 (+http://www.google.com/bot.html)
192.168.1.22     2026-09-01 00:11:21      2026-09-01      0      GET      /index.html      400      289      https://t.co      "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (K
HTML
192.168.1.85     2026-09-01 00:11:56      2026-09-01      0      PUT      /home      200      1653      https://t.co      "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML
192.168.1.76     2026-09-01 00:14:49      2026-09-01      0      DELETE   /index.html      301      3608      https://www.google.com      "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X)
AppleWebKit/605.1.15 (KHTML
10.0.0.15        2026-09-01 00:19:10      2026-09-01      0      GET      /api/v1/user      301      2048      direct      "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML
192.168.1.39     2026-09-01 00:19:43      2026-09-01      0      DELETE   /products      200      3039      direct      "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML
10.0.0.37        2026-09-01 00:30:12      2026-09-01      0      GET      /home      200      4853      direct      "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15 (KH
TML
192.168.1.17     2026-09-01 00:33:58      2026-09-01      0      GET      /home      200      2120      https://t.co      "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.
1.15 (KHTML
Time taken: 2.977 seconds, Fetched: 10 row(s)
hive>
```

Overall Flow

Hadoop → Start Hive → Create Database → Create Table → Load Data from HDFS →
Verify First 10 Rows → Hive Queries

3.HIVE QUERIES TO ANALYSE DATA

1. Daily Traffic Trend

- **Objective:** Track total web server request volume per day to identify traffic patterns and peak activity days.
- **Hive Query:**
- **Run:**

Output:

```
hive> select
> log_date,
> count(*) as total_requests
> from WebServerLogs
> GROUP BY log_date;
```

```
OK
2026-09-01      477
2026-09-02      528
2026-09-03      463
2026-09-04      465
2026-09-05      512
2026-09-06      522
2026-09-07      527
2026-09-08      482
2026-09-09      512
2026-09-10      512
Time taken: 121.563 seconds, Fetched: 10 row(s)
hive>
```

SELECT

log_date,

COUNT(*) AS total_requests

FROM WebServerLogs

GROUP BY log_date;

- **Explanation:** Groups records by date (log_date) and calculates total request counts to show server traffic over time.

2. Hourly Traffic Distribution

- **Objective:** Identify peak load hours on the web server to optimize server maintenance schedules and resource scaling.
- **Hive Query:**
- **Run:**

```
Time taken: 20.701 seconds, Fetched: 176 row(s)
hive> select
> hour,
> count(*) as total_requests
> from WebServerLogs
> GROUP BY hour;
```

Output:

```
Total MapReduce CPU Time Spent: 5 seconds 160 msec
OK
0      219
1      218
2      203
3      227
4      189
5      201
6      220
7      200
8      238
9      201
10     198
11     191
12     209
13     203
14     201
15     220
16     209
17     217
18     207
19     182
20     232
21     194
22     202
23     219
Time taken: 28.367 seconds, Fetched: 24 row(s)
hive>
```

SELECT

hour,

COUNT(*) AS total_requests

FROM WebServerLogs

GROUP BY hour;

- **Explanation:** Aggregates incoming requests across every hour of the day (0–23) to pinpoint high-volume hours.

3. Traffic Volume by Time of Day

- **Objective:** Categorize server traffic into broad time windows (Morning, Afternoon, Evening, Night) for high-level operational reporting.
- **Hive Query:**
- **Run:**

```
hive> select
> case
> when hour between 5 and 11 then 'Morning'
> when hour between 12 and 16 then 'Afternoon'
> when hour between 17 and 20 then 'Evening'
> else 'Night'
> END AS time_of_day,
> count(*) as total_requests
> FROM WebServerLogs
> GROUP BY
> case
> when hour between 5 and 11 then 'Morning'
> when hour between 12 and 16 then 'Afternoon'
> when hour between 17 and 20 then 'Evening'
> else 'Night'
> END;
Query ID = hdn_20760808084678_dda9bdf1_e563_4ff0_85cc_b977ed918163
```

Output:

```
OK
Afternoon      1042
Evening        938
Morning       1449
Night         1671
Time taken: 27.907 seconds, Fetched: 4 row(s)
hive>
```

```

SELECT
CASE
    WHEN hour BETWEEN 5 AND 11 THEN 'Morning'
    WHEN hour BETWEEN 12 AND 16 THEN 'Afternoon'
    WHEN hour BETWEEN 17 AND 20 THEN 'Evening'
    ELSE 'Night'
END AS time_of_day,
COUNT(*) AS total_requests
FROM WebServerLogs
GROUP BY
CASE
    WHEN hour BETWEEN 5 AND 11 THEN 'Morning'
    WHEN hour BETWEEN 12 AND 16 THEN 'Afternoon'
    WHEN hour BETWEEN 17 AND 20 THEN 'Evening'
    ELSE 'Night'
END;

```

- **Explanation:** Uses conditional CASE statements to group hours into four major time segments and count incoming requests for each.

4. Unique Visitor Request Volume

- **Objective:** Measure engagement levels by counting total requests made by individual IP addresses.
- **Hive Query:**
- **Run:**

```

Night 1671
Time taken: 66.477 seconds, Fetched: 1 row(s)
hive> select
  > ip_address as user_id,
  > count(*) as total_requests
  > from WebServerLogs
  > group by ip_address;
Query ID = hdp_20260909085709_9bf07439-0b9b-454a-bcea-684ef126a2f3

```

Output:

```

Total MapReduce CPU Time Spent: 5 seconds 230 msec
OK
10.0.0.1      32
10.0.0.10     28
10.0.0.11     32
10.0.0.12     32
10.0.0.13     33
10.0.0.14     34
10.0.0.15     23

```

```
SELECT  
  
ip_address AS user_id,  
  
COUNT(*) AS total_requests  
  
FROM WebServerLogs  
  
GROUP BY ip_address;
```

- **Explanation:** Groups server entries by client IP address (ip_address) to display individual user activity levels.

5. Page Popularity Analysis

- **Objective:** Identify the most and least visited URLs on the website to understand user navigation habits.
- **Hive Query:**

Run:

```
hive> select  
> request_url as page,  
> count(*) as total_requests  
> from WebServerLogs  
> group by request_url;  
Query ID = hdp_20260909092823_a6162fa0-1a78-4a07-9c06-bab0b8b71a36
```

Output:

```
OK  
/about 93  
/api/v1/user 260  
/cart 331  
/checkout 192  
/contact 151  
/home 771  
/images/logo.png 114  
/index.html 960  
/login 377  
/products 605  
/products/item1 357  
/products/item2 365  
/static/app.js 206  
/static/style.css 218  
Time taken: 39.964 seconds, Fetched: 14 row(s)  
hive>
```

```
SELECT  
  
request_url AS page,  
  
COUNT(*) AS total_requests  
  
FROM WebServerLogs  
  
GROUP BY request_url;
```

- **Explanation:** Counts hit volume for each distinct URL endpoint (request_url).

6. Raw User-Agent Distribution

- **Objective:** Determine the diversity of browsers and operating systems accessing the platform.

- **Hive Query:**

Run:

```
Time taken: 0.733 seconds, Fetched: 3 row(s)
hive> select
> user_agent,
> count(*) as total_requests
> from WebServerLogs
> GROUP BY user_agent;
Query ID = hdp_20260909093538_e2373a30-86be-4e96-801f-7af4151ac014
```

Output:

```
OK
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML 988
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML 990
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML 957
Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML 1024
Googlebot/2.1 (-http://www.google.com/bot.html) 1041
Time taken: 33.235 seconds, Fetched: 5 row(s)
hive>
```

SELECT

user_agent,

COUNT(*) AS total_requests

FROM WebServerLogs

GROUP BY user_agent;

- **Explanation:** Aggregates total requests by exact User-Agent client strings.

7. Device Type Breakdown

- **Objective:** Classify traffic into high-level device categories (Desktop, Mobile, Tablet, Bot/Crawler) to guide frontend UI/UX responsiveness.

- **Hive Query:**

Run:

```
hive> select
> case
> when LOWER(user_agent) LIKE '%bot%' OR LOWER(user_agent) LIKE '%spider%' THEN 'bot / crawler'
> WHEN LOWER(user_agent) LIKE '%mobile%' OR LOWER(user_agent) LIKE '%android%' OR LOWER(user_agent) LIKE '%iphone%' THEN 'Mobile'
> WHEN LOWER(user_agent) LIKE '%tablet%' OR LOWER(user_agent) LIKE '%ipad%' THEN 'Tablet'
> WHEN LOWER(user_agent) LIKE '%window%' OR LOWER(user_agent) LIKE '%linux%' OR LOWER(user_agent) LIKE '%macintosh%' OR LOWER(user_agent) LIKE '%x11%' THEN 'Desktop'
> ELSE 'unknown'
> END AS device_type,
> count(*) as total_requests
> from WebServerLogs
> GROUP BY
> case
> when LOWER(user_agent) LIKE '%bot%' OR LOWER(user_agent) LIKE '%spider%' THEN 'bot / crawler'
> WHEN LOWER(user_agent) LIKE '%mobile%' OR LOWER(user_agent) LIKE '%android%' OR LOWER(user_agent) LIKE '%iphone%' THEN 'Mobile'
> WHEN LOWER(user_agent) LIKE '%tablet%' OR LOWER(user_agent) LIKE '%ipad%' THEN 'Tablet'
> WHEN LOWER(user_agent) LIKE '%window%' OR LOWER(user_agent) LIKE '%linux%' OR LOWER(user_agent) LIKE '%macintosh%' OR LOWER(user_agent) LIKE '%x11%' THEN 'Desktop'
> ELSE 'unknown'
> END;
Query ID = hdp_20260909094328_7115623f-9537-441f-9f45-0b0e3fec8feb
```

Output:

```
OK
Desktop 2935
Mobile 1024
bot / crawler 1041
Time taken: 32.988 seconds, Fetched: 3 row(s)
hive>
```

```

SELECT

CASE

    WHEN LOWER(user_agent) LIKE '%bot%' OR LOWER(user_agent) LIKE '%spider%'
    THEN 'bot / crawler'

    WHEN LOWER(user_agent) LIKE '%mobile%' OR LOWER(user_agent) LIKE
'%android%' OR LOWER(user_agent) LIKE '%iphone%' THEN 'Mobile'

    WHEN LOWER(user_agent) LIKE '%tablet%' OR LOWER(user_agent) LIKE '%ipad%'
    THEN 'Tablet'

    WHEN LOWER(user_agent) LIKE '%window%' OR LOWER(user_agent) LIKE
'%linux%' OR LOWER(user_agent) LIKE '%macintosh%' OR LOWER(user_agent) LIKE
'%x11%' THEN 'Desktop'

    ELSE 'unknown'

END AS device_type,

COUNT(*) AS total_requests

FROM WebServerLogs

GROUP BY

CASE

    WHEN LOWER(user_agent) LIKE '%bot%' OR LOWER(user_agent) LIKE '%spider%'
    THEN 'bot / crawler'

    WHEN LOWER(user_agent) LIKE '%mobile%' OR LOWER(user_agent) LIKE
'%android%' OR LOWER(user_agent) LIKE '%iphone%' THEN 'Mobile'

    WHEN LOWER(user_agent) LIKE '%tablet%' OR LOWER(user_agent) LIKE '%ipad%'
    THEN 'Tablet'

    WHEN LOWER(user_agent) LIKE '%window%' OR LOWER(user_agent) LIKE
'%linux%' OR LOWER(user_agent) LIKE '%macintosh%' OR LOWER(user_agent) LIKE
'%x11%' THEN 'Desktop'

    ELSE 'unknown'

END;

```

- **Explanation:** Uses string matching (LIKE) on user_agent to categorize clients into Desktops, Mobiles, Tablets, and Automated Bots.

8. HTTP Status Code Breakdown

- **Objective:** Assess web application health by analysing the ratio of successful responses against client and server errors.

- **Hive Query:**

Run:

```
hive> select
> case
> when status_code between 200 and 299 THEN 'Success (2xx)'
> when status_code between 300 and 399 THEN 'Redirection (3xx)'
> when status_code between 400 and 499 THEN 'Client Error (4xx)'
> when status_code between 500 and 599 THEN 'Server Error (5xx)'
> ELSE 'UNKNOWN'
> END AS types_of_codes,
> count(*) as total_requests
> from WebServerLogs
> GROUP BY
> case
> when status_code between 200 and 299 THEN 'Success (2xx)'
> when status_code between 300 and 399 THEN 'Redirection (3xx)'
> when status_code between 400 and 499 THEN 'Client Error (4xx)'
> when status_code between 500 and 599 THEN 'Server Error (5xx)'
> ELSE 'UNKNOWN'
> END;
Query ID = hdp_20260909094549_b7823913-5f1e-4752-bc4a-2279cc087756
Total jobs = 1
```

Output:

```
OK
Client Error (4xx)      348
Redirection (3xx)      469
Server Error (5xx)      132
Success (2xx)          4051
Time taken: 33.345 seconds, Fetched: 4 row(s)
hive>
```

SELECT

CASE

WHEN status_code BETWEEN 200 AND 299 THEN 'Success (2xx)'

WHEN status_code BETWEEN 300 AND 399 THEN 'Redirection (3xx)'

WHEN status_code BETWEEN 400 AND 499 THEN 'Client Error (4xx)'

WHEN status_code BETWEEN 500 AND 599 THEN 'Server Error (5xx)'

ELSE 'UNKNOWN'

END AS types_of_codes,

COUNT(*) AS total_requests

FROM WebServerLogs

GROUP BY

CASE

WHEN status_code BETWEEN 200 AND 299 THEN 'Success (2xx)'

WHEN status_code BETWEEN 300 AND 399 THEN 'Redirection (3xx)'

WHEN status_code BETWEEN 400 AND 499 THEN 'Client Error (4xx)'

WHEN status_code BETWEEN 500 AND 599 THEN 'Server Error (5xx)'

ELSE 'UNKNOWN'

END;

- **Explanation:** Groups numeric HTTP response codes into standard status categories (2xx, 3xx, 4xx, 5xx).

9. Broken Links Analysis

- **Objective:** Detect broken links, missing assets, and unlinked endpoints by isolating HTTP 404 client errors per URL.
- **Hive Query:**

Run:

```
hive> select
  > request_url as page,
  > count(*) as total
  > from WebServerLogs
  > where status_code = 404
  > GROUP BY request_url;
Query ID = hdp_20260909094732_b169fc24-5f11-4338-a983-fbf19f23c827
```

Output:

```
OK
/about      8
/api/v1/user 12
/cart      13
/checkout   9
/contact    5
/home      35
/images/logo.png 5
/index.html 55
/login     19
/products  28
/products/item1 14
/products/item2 18
/static/app.js 15
/static/style.css 10
Time taken: 33.387 seconds, Fetched: 14 row(s)
hive>
```

SELECT

request_url AS page,

COUNT(*) AS total

FROM WebServerLogs

WHERE status_code = 404

GROUP BY request_url;

- **Explanation:** Filters records where status_code = 404 and counts occurrences per page.

10. Server Faults by Page

- **Objective:** Identify unstable backend pages throwing 5xx internal server errors to prioritize bug fixes and debugging.
- **Hive Query:**

Run:

```
hive> select
  > request_url as page,
  > count(*) as total
  > from WebServerLogs
  > where status_code between 500 and 599
  > GROUP BY request_url;
Query ID = hdp_20260909094925_09918666-2bc6-4aec-8f7e-8f2627bc665f
```


Output:

```
/about 1
/api/v1/user 4
/cart 11
/checkout 11
/contact 2
/home 25
/images/logo.png 2
/index.html 23
/login 7
/products 14
/products/item1 7
/products/item2 16
/static/app.js 5
/static/style.css 4
Time taken: 36.971 seconds, Fetched: 14 row(s)
hive>
```

SELECT

request_url AS page,

COUNT(*) AS total

FROM WebServerLogs

WHERE status_code BETWEEN 500 AND 599

GROUP BY request_url

ORDER BY total DESC;

- **Explanation:** Filters internal server error responses (500–599) and ranks affected URLs in descending order of failure frequency.

11. Resource Bandwidth Usage per IP

- **Objective:** Monitor network bandwidth consumption to flag heavy usage users or potential DDoS/scraping activity.
- **Hive Query:**

Run:

```
hive> select
> ip_address as user_id,
> sum(bytes_transferred) as resource_used
> from WebServerLogs
> GROUP BY ip_address;
Query ID = hdp_20260909095114_224a0cd5-2266-44d7-8286-10461ca2032f
Total taken: 5 seconds 960 msec
```

Output:

```
Total MapReduce CPU Time Spent: 5 seconds 960 msec
OK
10.0.0.1 70997
10.0.0.10 62838
10.0.0.11 56543
10.0.0.12 63515
10.0.0.13 74928
10.0.0.14 81276
10.0.0.15 53332
10.0.0.16 13000
```

SELECT

ip_address AS user_id,

SUM(bytes_transferred) AS resource_used

FROM WebServerLogs

GROUP BY ip_address;

- **Explanation:** Sums the total data transferred (bytes_transferred) grouped by client IP address.

Conclusion & Business Impact

- 1. Infrastructure Optimization:** Identifying peak hours and heavy traffic windows enables better auto-scaling policies.
- 2. Quality Assurance & Maintenance:** Pinpointing exact pages causing 404 and 5xx errors speeds up bug resolution.
- 3. Bandwidth Management:** Tracking resource usage by IP helps maintain system integrity and manage hosting costs.

4.PIG LATIN SETUP

To process and analyse the raw web server logs, Apache Pig was executed in MapReduce/Local interactive mode using Grunt shell. The step-by-step workflow for initializing the environment and executing the initial Pig Latin transformations is detailed below.

Initializing Apache Pig Interactive Shell

The interactive command-line interface (Grunt Shell) for Apache Pig was started to begin real-time Pig Latin script execution.

```
hdp@buntq-24:~$ pig -x local
2026-09-09 09:59:06,668 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-09-09 09:59:06,723 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-09 09:59:06,723 INFO pig.ExecTypeProvider: Picked LOCAL as the ExecType
2026-09-09 09:59:06,798 [main] INFO org.apache.pig.Main - Apache Pig version 0.18.0 (r1926484) compiled Jun 16 2025, 10:38:21
2026-09-09 09:59:07,119 [main] INFO org.apache.pig.Main - Logging error messages to: /home/hdp/pig_1788947946795.log
2026-09-09 09:59:06,854 [main] INFO org.apache.pig.Impl.util.Util - Default bootstrap file /home/hdp/pigbootstrap not found
2026-09-09 09:59:06,996 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.job.tracker.address
2026-09-09 09:59:07,001 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: file:///
2026-09-09 09:59:07,176 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
2026-09-09 09:59:07,176 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-default-c76392ac-2598-4832-9196-0478d4a6e1b7
2026-09-09 09:59:07,176 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
grunt>
```

Data Loading and Schema Definition

The dataset web_server_logs_5000.csv was loaded into Apache Pig using PigStorage(','). During loading, explicit data types were assigned to all fields (such as ip_address, timestamp, log_date, hour, status_code, and bytes_transferred) to ensure accurate downstream aggregation.

```
grunt> WebServerLogs_raw = LOAD '/home/hdp/Downloads/web_server_logs_5000.csv'
>> USING PigStorage(',')
>> AS (
>>   ip_address:chararray,
>>   timestamp:chararray,
>>   log_date:chararray,
>>   hour:int,
>>   http_method:chararray,
>>   request_url:chararray,
>>   status_code:int,
>>   bytes_transferred:long,
>>   referrer:chararray,
>>   user_agent:chararray
>> );
2026-09-09 10:00:54,327 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum
grunt>
```

Inspecting Initial Records

To verify that data parsing and field extraction succeeded without structural issues, the first 10 records of the relation were retrieved and printed to the terminal using the LIMIT operator along with DUMP.

```
2026-09-09 10:01:37,790 [main] INFO org.apache.pig.backend.hadoop.executionengine.MiniMapReduce - Total input pairs to process: 1
(10.0.0.11,2026-09-01 00:01:37,2026-09-01,0,GET,/static/style.css,200,4757,https://www.bing.com,"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML)
(10.0.0.15,2026-09-01 00:19:10,2026-09-01,0,GET,/api/v1/user,301,2048,direct,"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML)
(10.0.0.37,2026-09-01 00:30:12,2026-09-01,0,GET,/home,200,4853,direct,"Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML)
(172.16.0.20,2026-09-01 00:00:54,2026-09-01,0,GET,/products/item1,200,363,https://www.google.com,Googlebot/2.1 (+http://www.google.com/bot.html))
(192.168.1.17,2026-09-01 00:33:58,2026-09-01,0,GET,/home,200,2120,https://t.co,"Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML)
(192.168.1.22,2026-09-01 00:11:21,2026-09-01,0,GET,/index.html,400,289,https://t.co,"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML)
(192.168.1.39,2026-09-01 00:19:43,2026-09-01,0,DELETE,/products,200,3039,direct,"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML)
(192.168.1.76,2026-09-01 00:14:49,2026-09-01,0,DELETE,/index.html,301,3600,https://www.google.com,"Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15 (KHTML)
(192.168.1.78,2026-09-01 00:07:05,2026-09-01,0,GET,/products,200,2839,https://www.facebook.com,Googlebot/2.1 (+http://www.google.com/bot.html))
(192.168.1.85,2026-09-01 00:11:56,2026-09-01,0,PUT,/home,200,1653,https://t.co,"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML)
grunt>
```

Data Filtering, Sorting, and Output Dump

A conditional filter was applied to remove header entries and extract relevant log instances. The filtered dataset was then ordered and dumped directly to stdout to validate data accuracy prior to executing analytical queries.

```
grunt> WebServerLogs = FILTER WebServerLogs_raw BY ip_address != 'ip_address';
grunt> DESCRIBE WebServerLogs;
WebServerLogs: (ip_address: chararray,timestamp: chararray,log_date: chararray,hour: int,http_method: chararray,request_url: chararray,status_code: int,bytes_transferred: long,referrer: chararray,user_agent: chararray)
grunt> first_10_logs = LIMIT WebServerLogs 10;
grunt> DUMP first_10_logs;
1076 00:00:10.0140.744 [info] INFO: use apache-sql-tools: pipetools: ScriptState: ... Dis features used in the script: FILTER LIMIT
```

5.PIG LATIN SCRIPTS

Analysis 1: Daily Request Distribution (Day_vs_Req)

This script aggregates total server requests by calendar date (log_date) to monitor daily traffic trends, detect high-volume spikes, and evaluate server capacity needs over time.

- Pig Latin Logic: Groups the Webserver Logs relation by log_date and applies the COUNT() aggregate function to determine total request volume per day.
- Objective: Identify peak operational days and assess traffic stability across the log duration.

Run:

```
grunt> grp_date = GROUP WebServerLogs BY log_date;  
grunt> req_per_date = FOREACH grp_date GENERATE group AS log_date, COUNT(WebServerLogs) AS total_requests;  
grunt> DUMP req_per_date;
```

Output:

```
(2026-09-01,477)  
(2026-09-02,528)  
(2026-09-03,463)  
(2026-09-04,465)  
(2026-09-05,512)  
(2026-09-06,522)  
(2026-09-07,527)  
(2026-09-08,482)  
(2026-09-09,512)  
(2026-09-10,512)
```

Analysis 2: Hourly Request Distribution (Hour_vs_Req)

This script evaluates traffic volume across a 24-hour cycle (0 to 23) to identify peak load hours and off-peak maintenance windows for the web server infrastructure.

- Pig Latin Logic: Groups log entries by hour and computes the request count per individual hour of the day.
- Objective: Pinpoint precise hourly traffic surges to optimize server auto-scaling schedules.

Run:

```
grunt> grp_hour = GROUP WebServerLogs BY hour;  
grunt> req_per_hour = FOREACH grp_hour GENERATE group AS hour, COUNT(WebServerLogs) AS total_requests;  
grunt> DUMP req_per_hour;
```

Output:

```
(0,219)  
(1,218)  
(2,203)  
(3,227)  
(4,189)  
(5,201)  
(6,220)  
(7,200)  
(8,238)  
(9,201)  
(10,198)  
(11,191)  
(12,209)  
(13,203)  
(14,201)  
(15,220)  
(16,209)  
(17,217)  
(18,207)  
(19,182)  
(20,232)  
(21,194)  
(22,202)  
(23,219)  
grunt>
```

Analysis 3: Device Type Classification (device_vs_req)

This script categorizes incoming requests by device type—such as Mobile, Desktop, Tablet, and Bot/Crawler—by matching regular expressions against the HTTP user_agent field.

- Pig Latin Logic: Employs ternary conditional statements and LOWER(user_agent) MATCHES pattern matching to classify each entry into device buckets, followed by group aggregation.
- Objective: Understand user client demographics and measure automated bot traffic targeting the platform.

Run:

```
grunt> classified_device = FOREACH WebServerLogs GENERATE
grunt>   ((LOWER(user_agent) MATCHES '*bot.*' OR LOWER(user_agent) MATCHES '*spider.*') ? 'bot / crawler' :
>>   ((LOWER(user_agent) MATCHES '*mobile.*' OR LOWER(user_agent) MATCHES '*android.*' OR LOWER(user_agent) MATCHES '*iphone.*') ? 'Mobile' :
>>   ((LOWER(user_agent) MATCHES '*tablet.*' OR LOWER(user_agent) MATCHES '*ipad.*') ? 'Tablet' :
>>   ((LOWER(user_agent) MATCHES '*window.*' OR LOWER(user_agent) MATCHES '*linux.*' OR LOWER(user_agent) MATCHES '*macintosh.*' OR LOWER(user_agent) MATCHES '*x11.*') ? 'Desktop' : 'unknown')) AS device_type;
grunt>
grunt> grp_device = GROUP classified_device BY device_type;
grunt> req_per_device = FOREACH grp_device GENERATE group AS device_type, COUNT(classified_device) AS total_requests;
grunt> DUMP req_per_device;
```

Output:

```
(Mobile,1024)
/Desktop,2935)
(bot / crawler,1041)
grunt>
```

Time of Day Request Classification(Time_of_Day_vs_Req)

This script groups requests into broad temporal time blocks—Morning (05–11), Afternoon (12–16), Evening (17–20), and Night (21–04)—to evaluate user engagement patterns based on local time.

Run:

```
grunt> classified_time = FOREACH WebServerLogs GENERATE
grunt>   ((hour >= 5 AND hour <= 11) ? 'Morning' :
>>   ((hour >= 12 AND hour <= 16) ? 'Afternoon' :
>>   ((hour >= 17 AND hour <= 20) ? 'Evening' : 'Night')) AS time_of_day;
grunt>
grunt> grp_time = GROUP classified_time BY time_of_day;
grunt> req_per_time = FOREACH grp_time GENERATE group AS time_of_day, COUNT(classified_time) AS total_requests;
grunt> DUMP req_per_time;
```

Output:

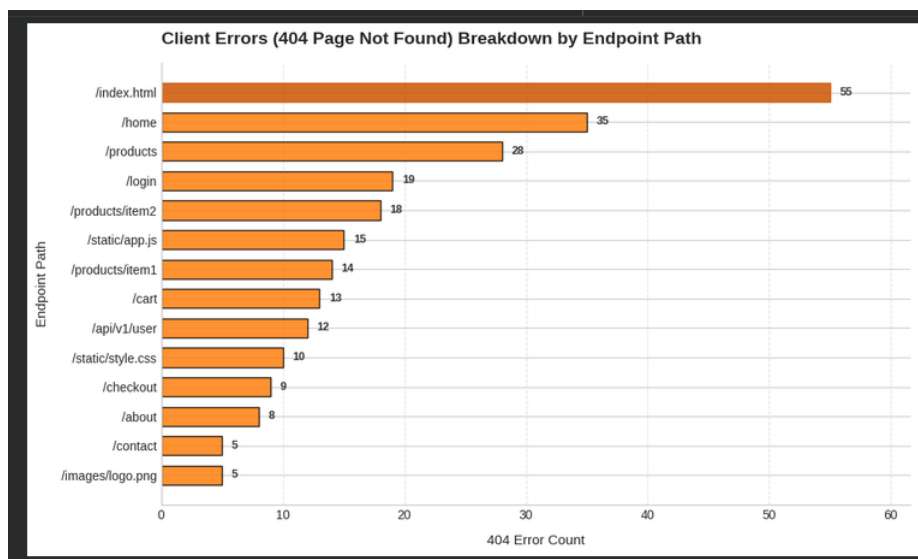
```
(Night,1671)
(Evening,838)
(Morning,1449)
(Afternoon,1042)
grunt>
```

6. Data Extraction & Visualizations

All visual analytical representations featured in the dashboard were generated programmatically using the Python Matplotlib library (along with Seaborn) to convert raw Apache Hive and Pig Latin aggregation results into intuitive charts.

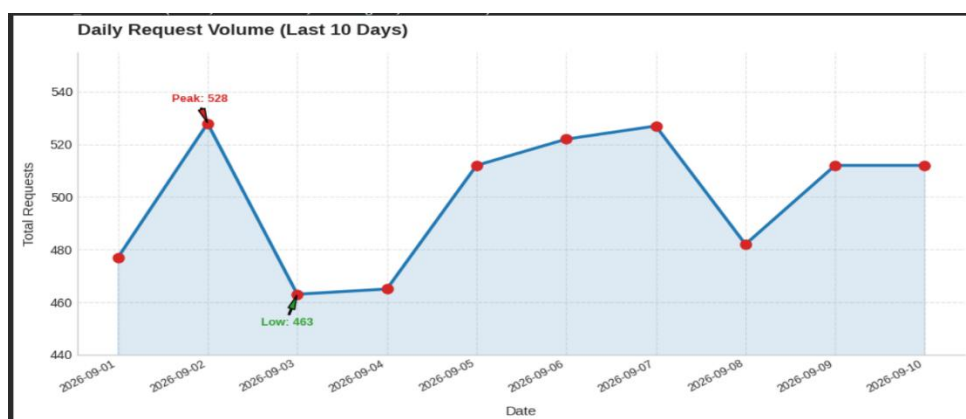
Client Errors vs Requested Page

- **Visualization Type:** Horizontal Bar Chart (Generated via Python Matplotlib)
- **Description:** Highlights specific request URLs generating HTTP 4xx errors (e.g., 404 Not Found). It identifies broken links and missing resource assets across the web application.



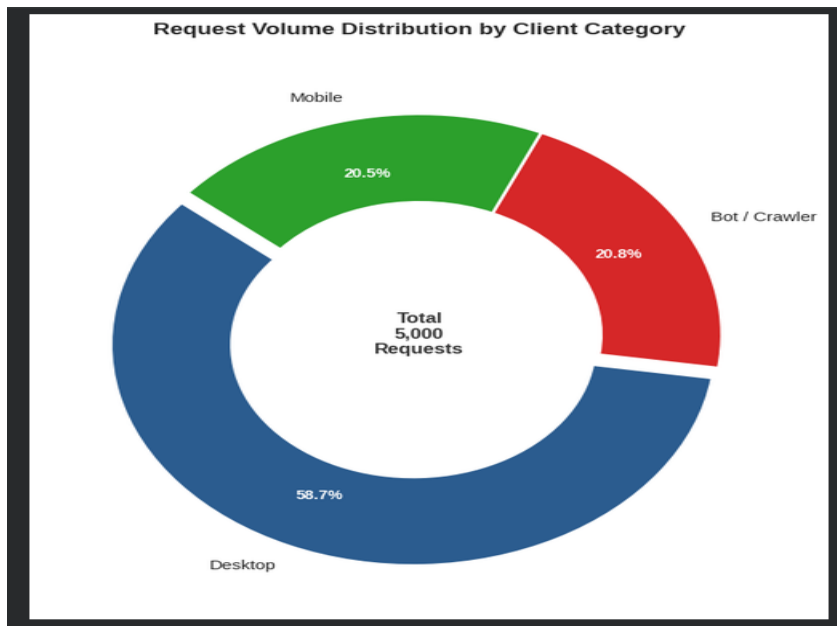
Daily Traffic Trends

- **Visualization Type:** Area / Line Chart (Generated via Python Matplotlib)
- **Description:** Plots overall server request volume mapped across individual days. It helps track daily traffic fluctuations, surge trends, and operational stability over time.



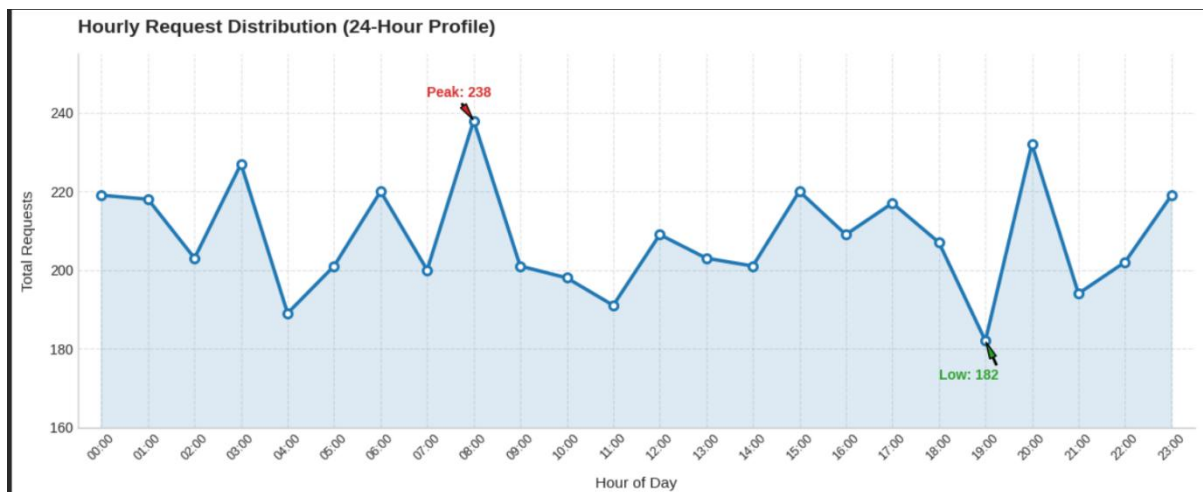
Device Type Breakdown

- **Visualization Type:** Donut / Pie Chart (Generated via Python Matplotlib)
- **Description:** Illustrates proportions of incoming web requests across device categories—including Desktop, Mobile, Tablet, and Bots/Crawlers.



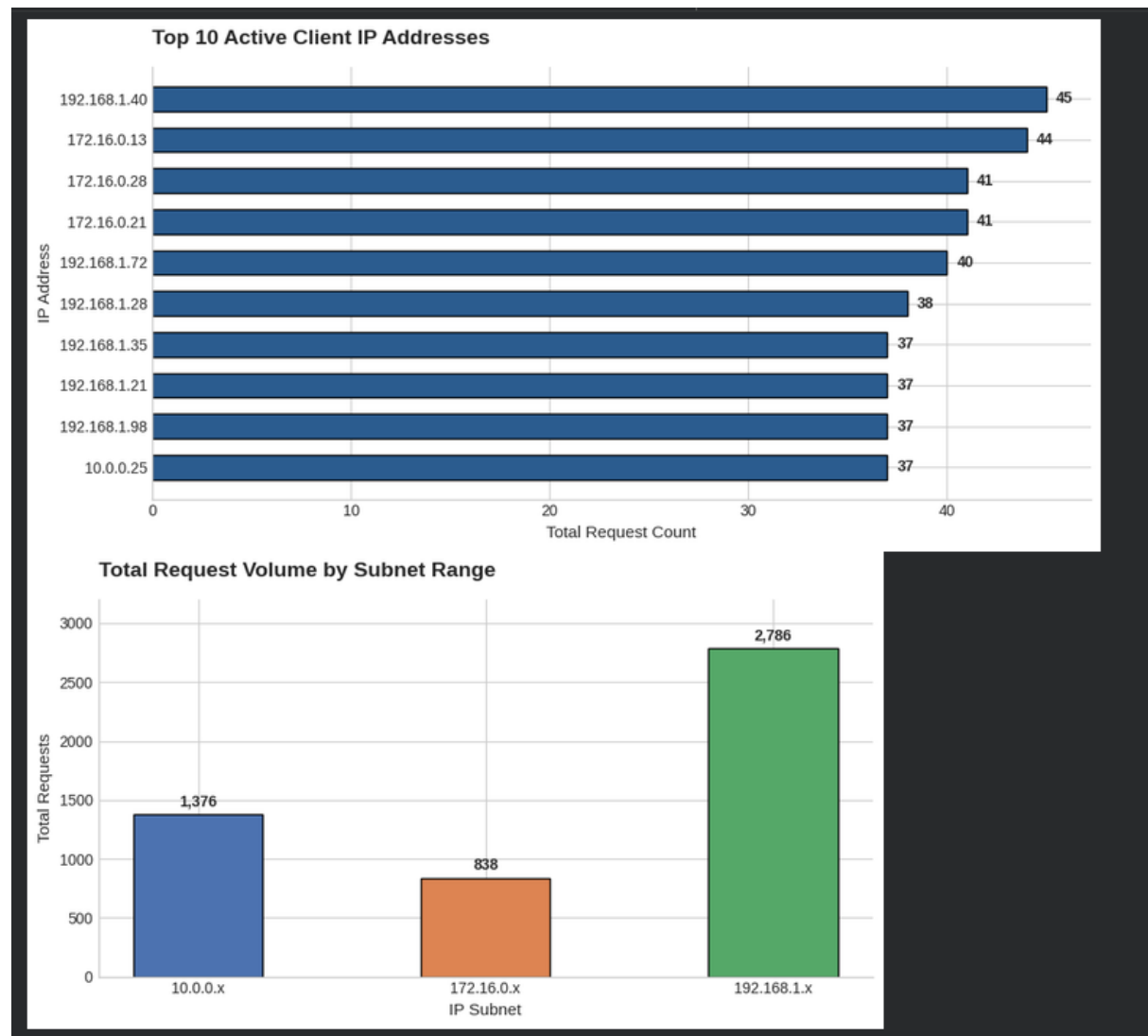
Hourly Request Distribution

- **Visualization Type:** Line Chart with Peak Annotations (Generated via Python Matplotlib)
- **Description:** Displays traffic activity over a 24-hour daily cycle (hours 0 through 23). This chart clearly highlights peak usage hours and low-traffic windows for scheduling maintenance.



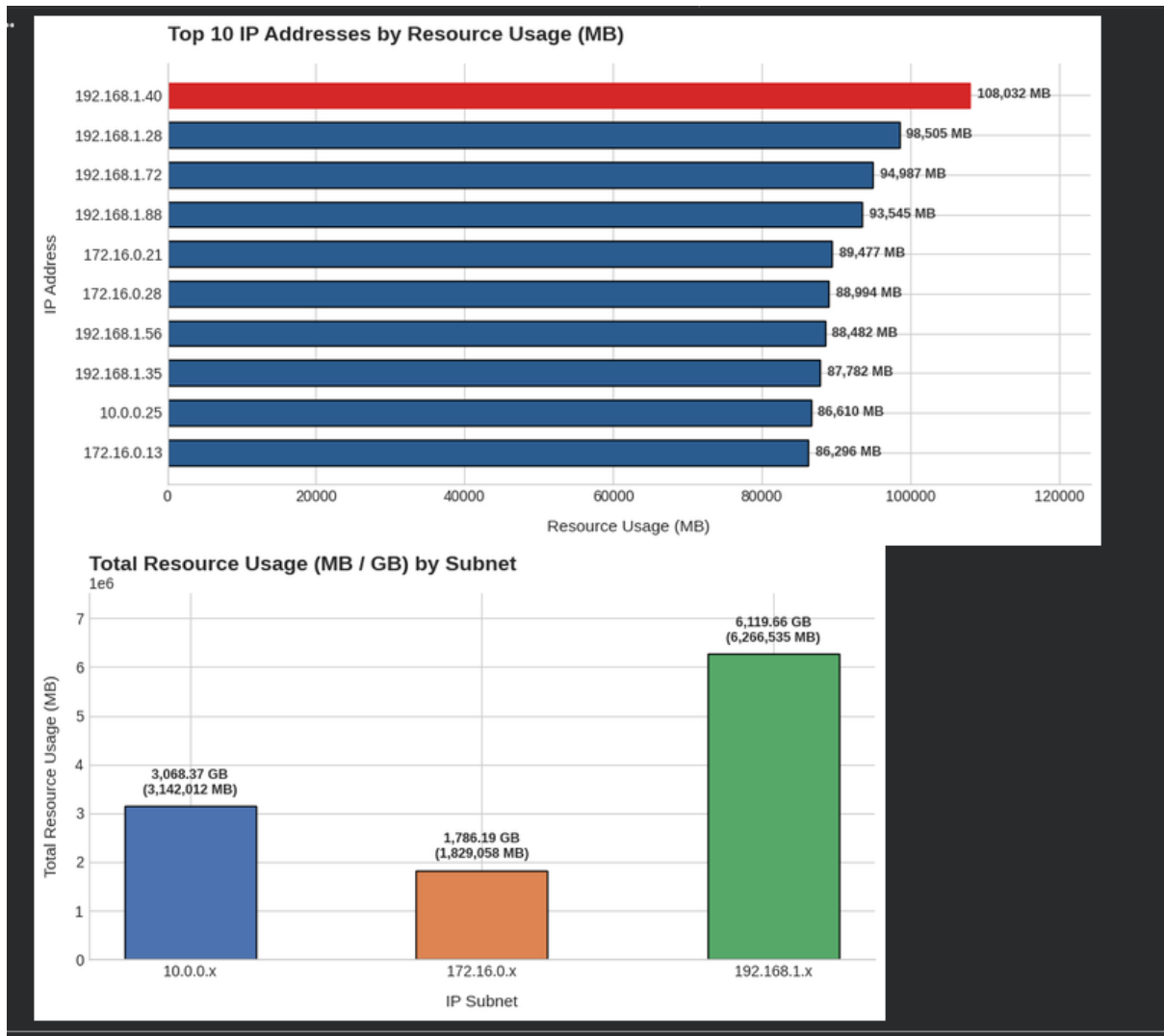
Top IP Addresses by Request Volume

- **Visualization Type:** Ranked Horizontal Bar Chart (Generated via Python Matplotlib)
- **Description:** Ranks client IP addresses by total request count. It helps administrators identify high-frequency users, potential automated scraper bots, or DDoS targets.

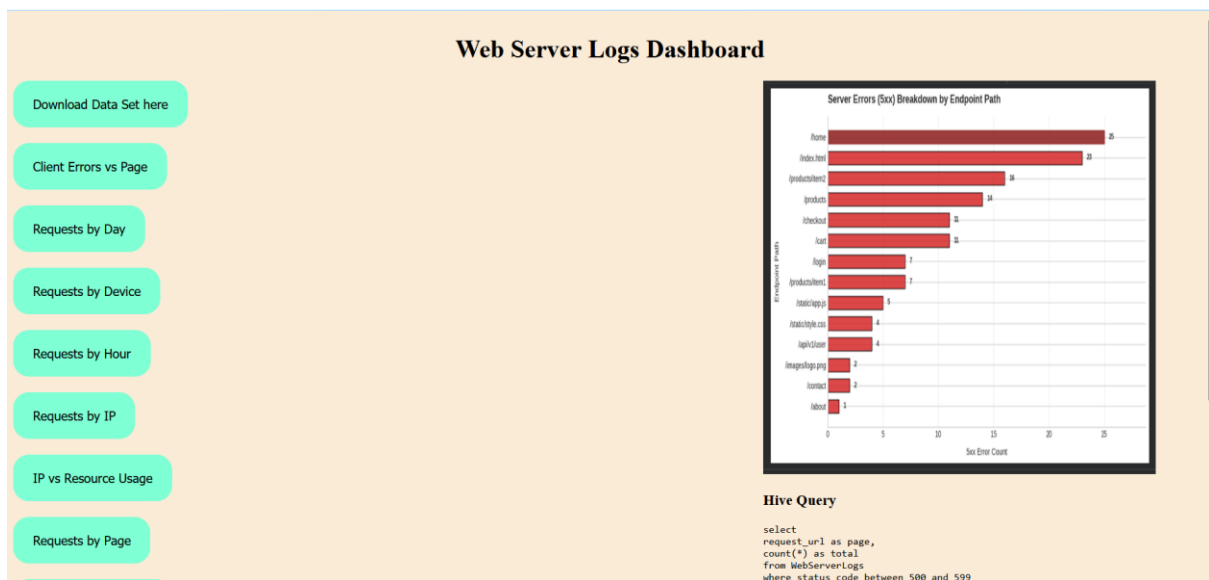
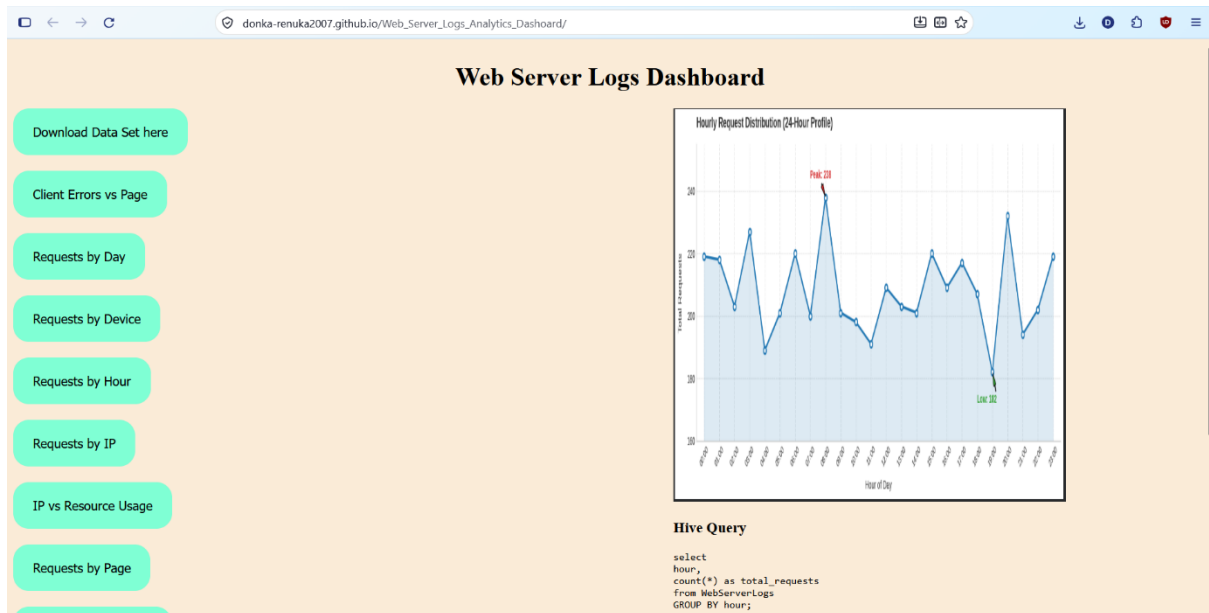


Visual 6: Resource Usage per IP

- **Visualization Type:** Bar Chart (Generated via Python Matplotlib)
- **Description:** Measures total bandwidth utilization (in bytes/MB) consumed by individual user IPs. It pinpoints heavy bandwidth consumers exceeding typical threshold limits.



7. Dashboard & Results



8. Project Setup Guide & GitHub Resources

1. Github repository for screenshots and scripts, queries, and visual images and guiding path:

https://github.com/donka-renuka2007/Web_Server_Logs_Analytics

2. Github repository for dashboard code:

https://github.com/donka-renuka2007/Web_Server_Logs_Analytics_Dashoard

3. Deployed link of Dashboard:

https://donka-renuka2007.github.io/Web_Server_Logs_Analytics_Dashoard/